Desenvolvimento de Sistemas Web II

HTTP API e microserviços

DIM0547 — Turma 01 · Aulas 03–04 (Sprint 0) · 31/08 e 02/09

Prof. Fernando · UFRN · 2026.2

Roteiro da semana

Segunda, 31/08 — Estrutura e serviço principal

Bloco	O que vemos
A estrutura do MUSI	Componentes, como conversam, as camadas em pacotes
O serviço principal	Ktor (Kotlin) e Quarkus (Java 25), parte por parte
Rotas · injeção · DTOs · cliente · erros · OpenAPI	O mesmo serviço, nos dois stacks

Quarta, 02/09 — Go e os serviços

Bloco	O que vemos
Go essencial	Structs, interfaces, erros e `context`
Os serviços Go	`services/busca` e `services/conciliacao`
Ambiente e CI	`mise`, Docker Compose e o pipeline

> A comparação de sintaxe Kotlin × Java já vimos. Aqui olhamos a arquitetura: onde cada parte mora e como as peças se ligam.

Onde paramos

Nas duas primeiras aulas fechamos a fronteira HTTP:

```
Recurso, método, status e erro em problem+json
Cache: Cache-Control, ETag e o 304
REST: as restrições de Fielding, nível 2 de Richardson
Clean Architecture: a regra de dependência
```

Hoje vemos essas ideias encarnadas no MUSI: primeiro a estrutura do projeto, depois o serviço principal que vocês vão escolher construir.

Onde estudar depois

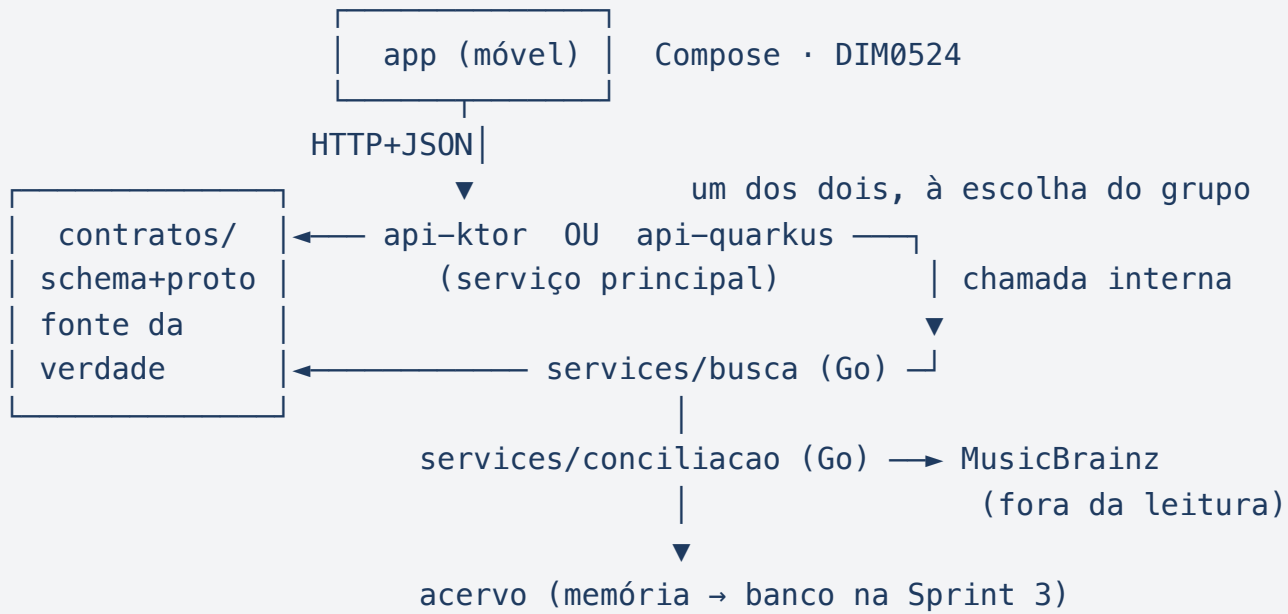
Fonte	Foco
Aprenda Go com Testes · playlist da turma	Go do zero, orientado a testes
Ktor · `ktor.io/docs` · Quarkus · `quarkus.io/guides`	O framework que o grupo escolher
Valente — *Engenharia de Software Moderna* · `engsoftmoderna.info`	Arquitetura e camadas

O material completo está nas referências do fim.

A estrutura do MUSI

Os componentes, e como conversam.

Os componentes



Cada componente é um processo com um papel. `contratos/` não roda: descreve o domínio que todos seguem.

> Hoje focamos no serviço principal (a caixa do meio). Quarta descemos aos serviços Go.

O papel de cada um

Componente	Faz	Não faz
`app`	Interface móvel (DIM0524)	Regra de negócio pesada
`api-ktor` / `api-quarkus`	Fronteira pública: HTTP, validação, orquestração	Ficar acoplado a um serviço
`services/busca`	A busca no acervo	Falar com terceiros
`services/conciliacao`	Casar obras com o MusicBrainz, em lote	Rodar no caminho da leitura
`contratos/`	Schema JSON e `.proto`	Depender de linguagem

> As duas APIs respondem a mesma coisa, contra o mesmo serviço Go. É a comparação Java × Kotlin, lado a lado, no mesmo produto.

Sequência: uma busca

Do cliente ao acervo e de volta, uma leitura:



A API traduz a query numa árvore de filtro, delega ao Go e devolve com os cabeçalhos de cache da aula 02.

A estrutura de pastas

```
musi/
├── api-ktor/       Kotlin · Ktor · Koin     → serviço principal (opção A)
├── api-quarkus/    Java 25 · Quarkus · CDI  → serviço principal (opção B)
├── services/       Go                        → busca e conciliação
├── contratos/      JSON Schema + .proto      → a fonte da verdade
├── shared/         domínio em Kotlin (KMP)   → usado por api-ktor e pelo app
├── http/           coleções para testar as APIs
├── docs/           ADRs, domínio, guias
├── mise.toml · docker-compose.yml · .github/workflows/ci.yml
```

> Abram o MUSI no IDE e sigam junto. Vocês escolhem um serviço principal, não os dois — o MUSI mantém ambos para servir às duas trilhas.

As camadas viram pacotes

A regra de dependência da aula 02 vira estrutura de diretórios, a mesma nos dois stacks:

```
api-ktor/.../                    api-quarkus/.../
  dominio/       ← entidades       dominio/       ← Obra, Filtro
  aplicacao/     ← casos de uso    aplicacao/     ← BuscarObras
  adaptadores/   ← HTTP, cliente Go  adaptadores/  ← HTTP, cliente Go
```

O `dominio` não importa HTTP nem banco. No Kotlin, o domínio vem do módulo `shared/`; no Java, é reescrito em `dominio/` — ver ADR-0001.

> A arquitetura é a mesma; só muda a linguagem e o framework. É por isso que a decisão entre os stacks é sobre a equipe e o domínio, não sobre "qual é melhor".

`contratos/`: a fonte da verdade

O que impede as duas APIs (e o Go) de divergirem não é disciplina: são os arquivos de `contratos/`.

Arquivo	Papel
`obra.schema.json`, `filtro.schema.json`	O formato JSON na fronteira HTTP
`exemplos/casos-de-busca.json`	Casos com resultado esperado
`musi/busca/v1/busca.proto`	O contrato gRPC (Sprint 2)

Os testes das três linguagens carregam os mesmos casos. Se discordarem, o CI acusa.

> O domínio existe em Kotlin, Java e Go. `contratos/` é o acordo entre eles — a decisão está registrada na ADR-0002.

O serviço principal

Ktor e Quarkus, parte por parte.

Duas implementações, uma decisão

O mesmo serviço, dois caminhos. Vocês escolhem um e justificam.

Parte	api-ktor (Kotlin)	api-quarkus (Java 25)
Rotas	código, numa árvore	anotações numa classe
Injeção de dependência	Koin, em execução	CDI, na compilação
Cliente do serviço Go	classe com `HttpClient`	interface declarativa
OpenAPI	plugin, do código	anotações, automático
Domínio	de `shared/`	reescrito em `dominio/`

> Não há escolha certa. A qualidade da justificativa é o que a rubrica avalia. Vamos ver cada linha desta tabela no código.

Rotas no Ktor: a árvore é código

A árvore de rotas inteira cabe numa tela e se lê de cima para baixo:

```
routing {
    get("/health") { call.respond(mapOf("status" to "UP")) }

    route("/obras") {
        get(Doc.buscaSimples)  { /* faceta pela query string */ }
        post(Doc.buscaComposta) { /* árvore de filtro no corpo */ }
    }
}
```

- Cada rota é código; não há anotação espalhada por classes
- O `Doc.buscaSimples` é o bloco de OpenAPI, mantido à parte (`RotasDoc.kt`)

> `api-ktor/.../adaptadores/web/Rotas.kt`. O controller traduz HTTP em chamada de caso de uso, e nada mais.

Rotas no Quarkus: anotações numa classe

O mesmo endpoint, declarado por anotações que o Quarkus varre na compilação:

```
@Path("/obras")
@Produces(MediaType.APPLICATION_JSON)
public class ObraResource {

    @GET
    public Response porFaceta(@QueryParam("dimensao") String d,
                              @QueryParam("valor") String v) { ... }

    @POST
    public List<ObraDto> buscar(FiltroDto dto) { ... }
}
```

`api-quarkus/.../adaptadores/web/ObraResource.java`. As anotações também alimentam o OpenAPI, sem código extra. É a mesma rota; muda como ela é declarada.

Injeção de dependência: quando o nó é dado

As duas resolvem a mesma dependência (`BuscarObras`), em momentos diferentes:

```
// Ktor + Koin: resolvido em execução
val buscarObras by inject<BuscarObras>()
// no módulo:  single { BuscarObras(get()) }
```

```
// Quarkus + CDI: verificado na compilação
@Inject
public ObraResource(BuscarObras buscarObras) { ... }
```

> Koin monta o grafo quando a aplicação sobe; o CDI monta na compilação e falha o build se faltar um nó. Duas filosofias, o mesmo resultado.

O caso de uso e a porta — iguais nos dois

O coração é idêntico: um caso de uso que depende de uma porta, não de HTTP.

```
interface FonteDeObras {           // a porta (aplicação)
    buscar(filtro): List<Obra>
}

class BuscarObras(fonte: FonteDeObras) {   // o caso de uso
    operator fun invoke(filtro) = fonte.buscar(filtro)
}
```

`BuscarObras` não sabe se a fonte é o serviço Go, um banco ou uma lista em memória.

> É a inversão de dependência da Clean Architecture: o caso de uso define a porta; o adaptador a implementa. O mesmo desenho nos dois stacks.

O cliente do serviço Go

O adaptador que implementa `FonteDeObras` falando com o Go — cada stack a seu modo:

```
// Ktor: uma classe que usa o HttpClient (o MESMO que o app usa)
class BuscaHttp(val cliente: HttpClient, val url: String) : FonteDeObras {
    override suspend fun buscar(filtro) =
        cliente.post("$url/buscar") { setBody(filtro.paraDto()) }.body()
}
```

```
// Quarkus: você declara a interface; o framework gera a chamada
@RegisterRestClient(configKey = "busca")
interface ClienteBusca { @POST List<ObraDto> buscar(FiltroDto f); }
```

No Ktor a chamada é escrita à mão; no Quarkus, declarada e gerada. `application.properties` aponta o `busca.url` para `http://localhost:9090`.

DTOs na fronteira

Nenhum dos dois serializa o domínio direto: um DTO faz a tradução na borda.

```
JSON  ──►  FiltroDto / ObraDto  ──►  Filtro / Obra   (paraDominio)
Obra  ──►  ObraDto              ──►  JSON            (paraDto)
```

- Espelham `contratos/*.schema.json`
- Isolam o formato público da modelagem interna: renomear um campo do domínio não quebra os clientes

> `Dtos.kt` e `Dtos.java`. É o mesmo padrão nos dois lados, e também no serviço Go — cada linguagem tem seu DTO, o contrato é um só.

Erros que o cliente consegue tratar — RFC 9457

Exceção do domínio vira `application/problem+json`, nos dois stacks:

```
// Ktor: um plugin StatusPages central
install(StatusPages) {
    exception<IllegalArgumentException> { call, causa ->
        call.respond(422, Problema(type=..., title="Filtro inválido", ...))
    }
}
```

```
// Quarkus: um ExceptionMapper
@Provider
class FiltroInvalidoMapper implements ExceptionMapper<IllegalArgumentException> { ... }
```

Um `400` sem corpo obriga quem consome a adivinhar. O `type` é a URI da classe do erro; o `detail`, aquela ocorrência. O serviço Go emite o mesmo formato.

Cache, do lado do servidor

O assunto da aula 02, agora declarado no serviço principal:

```
// Ktor: plugins cuidam de Cache-Control, ETag e do 304
install(CachingHeaders) { options { _, _ -> CachingOptions(maxAge = 60s) } }
install(ConditionalHeaders)   // ETag e requisição condicional
```

```
// Quarkus: Cache-Control montado na resposta
var cache = new CacheControl(); cache.setMaxAge(60);
return Response.ok(obras).cacheControl(cache).build();
```

`GET` declara `Cache-Control` e `ETag`; a escrita concorrente é protegida por `If-Match`. É o que a rúbrica da Sprint 1 verifica com `curl -v`.

OpenAPI e Swagger nos dois

A documentação da API, gerada — para explorar sem sair do navegador:

	`api-ktor`	`api-quarkus`
De onde vem	plugin, a partir das rotas	anotações, automático
Spec	`/openapi.json`	`/q/openapi`
Swagger UI	`/swagger`	`/q/swagger-ui`

> No Quarkus, o OpenAPI sai das anotações sem código. No Ktor, um plugin gera o spec a partir das rotas. A pasta `http/` traz coleções de Bruno e Postman para testar sem `curl`.

Como escolher entre Ktor e Quarkus

A decisão é da equipe, e precisa de justificativa:

Pesa a favor do Ktor	Pesa a favor do Quarkus
Time quer explorar Kotlin e corrotinas	Time consolida fundamentos da JVM
Preferência por rotas como código	Preferência por convenção e anotações
Reuso do domínio `shared/` com o app	Ecossistema Java maduro, CDI, GraalVM

> Registrem a escolha e o porquê na proposta (seção 5). Não há resposta certa; o que vale é a qualidade da justificativa.

"Mas o Ktor ficou mais verboso…"

Verdade — e vale contar onde as linhas moram. O serviço principal do MUSI, sem o domínio, para comparar igual:

Preocupação	api-ktor	api-quarkus
Rotas + OpenAPI	`Rotas.kt` + `RotasDoc.kt` = **154**	`ObraResource.java` = **64**
Bootstrap + injeção	`Aplicacao.kt` + `Modulos.kt` = **135**	`application.properties` + anotações
DTOs · caso de uso · cliente do Go	**mais curtos** que o Java	records + switch + interface

Total, sem o domínio: `api-ktor` ~438 · `api-quarkus` ~282.

> A diferença não é o Kotlin: é o Ktor. Fora da configuração, o Kotlin sai menor — DTOs, caso de uso e cliente são mais curtos que em Java.

Explícito ou por convenção: a troca

O mesmo comportamento, em lugares diferentes:

	Ktor (explícito)	Quarkus (convenção)
Plugins	`install(...)` visível em `Aplicacao.kt`	extensões automáticas
Injeção	grafo à mão em `Modulos.kt` (runtime)	`@Inject` / CDI (compilação)
OpenAPI	plugin, a partir do código	anotações, custo zero

- Ktor: mais linhas, mas o comportamento se lê num lugar, sem mágica; um `single` faltando vira teste (`ModulosTest.verify()`) no CI
- Quarkus: menos linhas, mas você confia nas convenções; em troca, a injeção é verificada na compilação

> A verbosidade do Ktor é o preço da explicitude: "o que está ligado está escrito". A brevidade do Quarkus é o preço da mágica. Nenhum dos dois está errado, e a escolha vai depender de vários fatores.

Uso de memória

Footprint medido no MUSI (RSS em repouso, mesma máquina):

Serviço	RSS idle	Startup
Go	~11 MB	<1 s
Quarkus nativo	~47 MB	0,028 s
Quarkus JVM	~113 MB	~1 s
Ktor	~199 MB	~2 s

DI em compilação (Quarkus) e AOT nativo pesam menos que DI em runtime (Ktor) sobre a JVM.

> Não é ranking — é a mesma troca dos slides de verbosidade, agora medida em MB. A T4 pede para **justificar** a escolha; aqui vai mais um eixo. Números e método em `docs/BENCHMARK.md`.

Exercício — explorar o serviço principal em grupo

Com o MUSI aberto e `docker compose up` rodando:

1. Escolham um stack (api-ktor ou api-quarkus) e o abram no IDE
2. Sigam uma requisição: rota → caso de uso → porta → cliente do Go
3. Abram o Swagger (/swagger ou /q/swagger-ui) e façam uma busca
4. Achem onde o erro vira problem+json, e onde o Cache-Control é setado
5. Comparem, no outro stack, como cada parte é declarada

No fim, cada grupo diz qual stack escolheria para o próprio projeto, e por quê.

Quarta · 02/09

Go e os serviços

A sintaxe de Go, junto com os dois serviços do MUSI.

Por que Go para os serviços

Go foi desenhado para serviços de rede: compila rápido, gera um binário único e trata concorrência como recurso de primeira classe.

Bom caso para Go	Onde outra escolha cabe
Trabalho limitado por rede ou por taxa	Regras de negócio ricas e ramificadas
Concorrência alta, muitos pedidos	Domínio que se beneficia de união etiquetada
Serviço pequeno, foco único	Time que já domina outra pilha

No MUSI há dois: a **busca** (leitura rápida) e a **conciliação** (lote, limitada a 1 req/s ao MusicBrainz).

> O serviço Go do MUSI não tem nenhuma dependência externa — só a biblioteca padrão. Por isso nem existe `go.sum`.

Go essencial: structs e funções

```
type Obra struct {
    ID      string   `json:"id"`
    Titulo  string   `json:"titulo"`
    Facetas []Faceta `json:"facetas"`
}

func dividir(a, b int) (int, error) {
    if b == 0 {
        return 0, fmt.Errorf("divisão por zero")
    }
    return a / b, nil
}
```

- `struct` agrupa campos; maiúscula inicial significa exportado
- Go não tem exceção: o erro é um valor, devolvido junto com o resultado

> `if err != nil` é o ritmo do Go. O caminho de erro fica visível na leitura, não escondido num `catch` distante.

Go essencial: interfaces e o `switch` de tipo

```
type Filtro interface { isFiltro() }          // fecha a hierarquia

type Tem struct{ Dimensao, Valor string }
func (Tem) isFiltro() {}

func Satisfaz(o Obra, f Filtro) bool {
    switch t := f.(type) {
    case Tem: /* ... */
    case Ate: return o.Ano <= t.Ano
    default:  panic(fmt.Sprintf("filtro não tratado: %T", f))
    }
}
```

O método não exportado `isFiltro()` é o mais perto de `sealed` que Go tem. O `default` existe porque o compilador de Go não verifica exaustividade — nos stacks Kotlin e Java, ele não aparece.

> É a mesma árvore de filtro, na terceira linguagem. Comparar essa escolha entre as três é conteúdo da disciplina.

Go essencial: `context`

Todo serviço precisa saber quando desistir. O `context.Context` carrega prazo e cancelamento, e é o primeiro parâmetro por convenção:

```
func buscar(ctx context.Context, f Filtro) ([]Obra, error) {
    select {
    case <-ctx.Done():
        return nil, ctx.Err()      // cliente desistiu ou estourou o prazo
    default:
    }
    // ... segue a busca
}
```

O `net/http` já entrega um `ctx` por requisição (`r.Context()`); a conciliação usa o dele para respeitar o cancelamento no meio de um lote longo.

> Sem `context`, uma chamada externa lenta trava a requisição inteira.

O serviço de busca: `services/busca`

Um servidor HTTP com a biblioteca padrão — sem framework:

```
func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/buscar", buscar)
    mux.HandleFunc("/health", health)

    srv := &http.Server{
        Addr: ":9090", Handler: mux,
        ReadHeaderTimeout:   5 * time.Second,
        MaxHeaderValueCount: 100,   // Go 1.27: limita cabeçalhos abusivos
    }
    srv.ListenAndServe()
}
```

`services/cmd/servidor/main.go`. O roteador (`ServeMux`) e o servidor já vêm na linguagem. O servidor declara seus limites.

O handler da busca

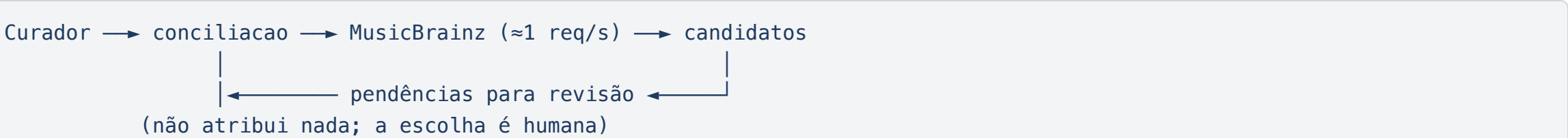
Traduz HTTP em chamada ao domínio: decodifica, chama `dominio.Buscar`, codifica.

```
func buscar(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        problema(w, 405, "metodo", "Use POST."); return
    }
    var dto filtroDTO
    if err := json.NewDecoder(r.Body).Decode(&dto); err != nil {
        problema(w, 400, "json-invalido", "Corpo inválido."); return
    }
    w.Header().Set("Cache-Control", "public, max-age=60")
    json.NewEncoder(w).Encode(dominio.Buscar(acervo, dto.paraDominio()))
}
```

Reconhecem os cabeçalhos e o `problem+json`? É a mesma fronteira HTTP das aulas 01–02, agora do lado Go. Em Go 1.27 o `encoding/json` foi reescrito e ficou mais rápido, sem mudar código.

O serviço de conciliação: `services/conciliacao`

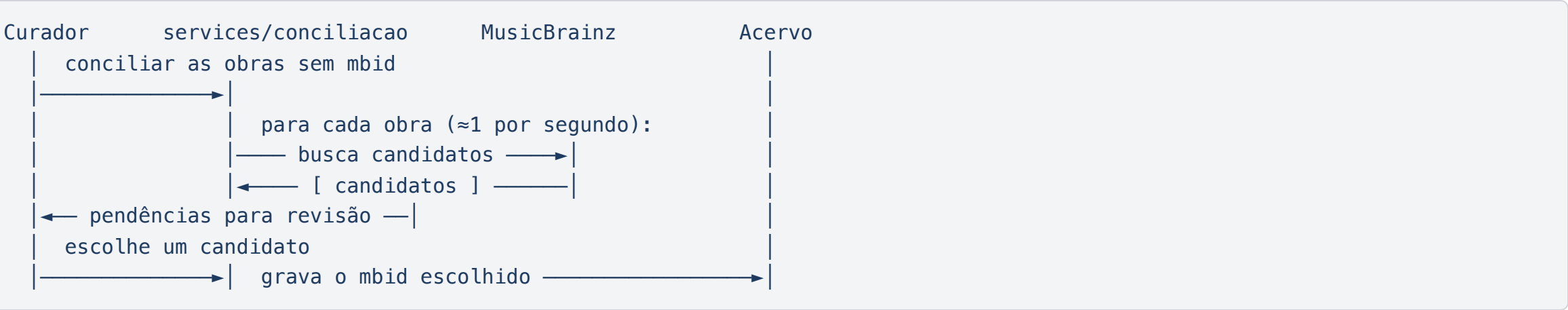
O segundo serviço Go, e o caso em que Go se justifica por característica — ADR-0003.



Peça	Onde, no IDE
Porta `CatalogoExterno` (o domínio não conhece o MusicBrainz)	`conciliacao/conciliacao.go`
Cliente com limite de 1 req/s	`conciliacao/musicbrainz.go`
Testes sem rede (fake em memória)	`conciliacao/conciliacao_test.go`

> Mesma porta-e-adaptador do serviço principal: o núcleo não conhece a rede; um adaptador a implementa. Limitado por taxa, tolerante a falha parcial, fora do ciclo de leitura.

Sequência: a conciliação



O serviço só coleta e apresenta; quem decide o vínculo é uma pessoa. Automatiza-se o trabalho de rede, não o trabalho de curadoria, que por sua vez permanece revisado por humanos.

Testes de contrato, carregados

Os testes das três linguagens carregam os mesmos arquivos de `contratos/exemplos/`:

```
contratos/exemplos/casos-de-busca.json   ◄── único ponto de verdade
      ▲           ▲           ▲
     Go          Java       Kotlin       carregam o mesmo arquivo
```

- Um caso novo no JSON alcança as três de uma vez
- Se discordarem, o CI acusa — a concordância virou garantia, não convenção
- O domínio Go é puro: `arch-go` no CI recusa um import de `net/http` ali

> Abram `dominio_test.go`, `DominioTest.java` e `CasosCompartilhadosTest.kt`: três linguagens, o mesmo arquivo de casos.

Ambiente reproduzível: `mise`

O `mise.toml` fixa a versão de cada ferramenta (Go 1.27, Java 25…) e dá nome às tarefas, para o mesmo comando rodar na sua máquina e no CI:

```
mise run build      # constrói os stacks
mise run test       # roda os testes
mise run lint:go    # go vet + gofmt + arch-go
```

E o Docker Compose sobe tudo com um comando:

```
docker compose up --build   # api-ktor (8080) · api-quarkus (8081) · busca (9090)
```

> Com a versão declarada, "funciona na minha máquina" deixa de ser um problema: o ambiente está no arquivo, e o CI usa as mesmas tarefas. No Codespaces as ferramentas já vêm prontas — você sobe tudo com `docker compose up`, sem instalar nada.

CI: o portão de qualidade

O workflow roda a cada push e PR, um job por componente, e um job final que só passa se todos passarem:

```
push / PR ──► kotlin · java · go · contratos · proto · docs ──► ci ✓
```

- O build roda em todo componente, sem filtro de caminho: um filtro deixaria passar uma quebra num componente que você não tocou
- A entrega de vocês precisa do essencial: build dos dois stacks (principal e Go), verde em `main`

> `.github/workflows/ci.yml`. Comecem simples: um job que compila o serviço principal e um que compila o Go. Refinam nas próximas sprints.

As tarefas da Sprint 0

O enunciado de cada entrega está em `docs/SPRINT-0-TAREFAS.md` — uma tarefa por cartão, com *pronto quando* e o peso na rubrica.

Ajuste recente	Detalhe
Backlog: mínimo 5 histórias (antes 10)	≥ 3 estimadas, todas priorizadas
`docs/proposta.md`: até 5 páginas (antes 3)	as 8 seções do guia

> A entrega vale metade da nota da Sprint 0 na estrutura do monorepo e no CI passando; o resto é a proposta, a decisão do stack e a divisão com Go.

Próximos passos

Até a entrega (11/09)

- Monorepo público com a estrutura, `mise run build` / `test` passando
- `docker compose up` sobe o serviço principal e o Go
- CI verde nos dois stacks
- Proposta com a decisão do stack (Ktor ou Quarkus) e a divisão com Go, e o backlog (≥ 5 histórias)

> 09/09 é encontro online, no horário da aula, para dúvidas sobre o pipeline e o ambiente.

Referências da semana

Go

- **Aprenda Go com Testes** — playlist da turma no YouTube · exercícios em `github.com/classrooms-fmarquesfilho/aprenda-go-com-testes`
- *A Tour of Go* · `go.dev/tour` · *Effective Go* · `go.dev/doc/effective_go`
- *Go by Example* · `gobyexample.com` · `net/http`, `encoding/json`, `context` · `pkg.go.dev`

Ktor, Quarkus e ambiente

- Ktor · `ktor.io/docs` · Quarkus · `quarkus.io/guides`
- `mise` · `mise.jdx.dev` · Docker Compose · `docs.docker.com/compose` · GitHub Actions · `docs.github.com/actions`

Disciplina e projeto de exemplo

- `github.com/fmarquesfilho/web2-2026-2`
- MUSI · `github.com/fmarquesfilho/musi` — `api-ktor/`, `api-quarkus/`, `services/`, `contratos/`